Answer 1

Remember, an 8-bit twos complement number has a range from -2^7 to 2^7-1, or -128 to 127.

The sums can be performed as follows:

0	0^1	0^1	1^1	0^1	1^1	1	1	(23)
0	0	1	0	1	1	1	0	(46)
0	1	0	0	0	1	0	1	(69)

1^1	1^1	1^1	1	0	1	0	0	(−12)
0	1	0	1	1	0	1	0	(90)
0	1	0	0	1	1	1	0	(78)

First, convert 63 (00111111) to −63 (11000001)

0	0	1	0	0	1	0^1	1	(37)
1	1	0	0	0	0	0	1	(−63)
1	1	1	0	0	1	1	0	(−26

First, convert 32 (00100000) to -32 (11100000), and
62 (00111110) to -62 (11000010)

1^1	1	1	0	0	0	0	0	(−32)
1	1	0	0	0	0	1	0	(−62)
1	0	1	0	0	0	1	0	(−94)

Q1(b)

An overflow can only occur when adding two numbers with the same sign. If the sign of the result is different to the sign of the operands, an overflow has occurred. Overflow does not occur when adding numbers with different signs, though it can occur when subtracting numbers with different signs (same rule as above, once converted to addition).

e.g. Given A, B, Sum defined as 16 bit numbers ([15:0]), then

```
assign {Co, Sum} = A + B;
```

Overflow (Ovf) can be determined as:

```
assign
    Ovf=(A[15]==B[15])?((sum[15]==A[15])?1'b0:1'b1):1'b0;
```

Answer 2

The integer part varies from -6 to +4 (but need to consider -7 to +4.something) so we require 4 integer bits, leaving 6 fractional bits.

We can follow the method given previously, for k fractional bits:

- Multiply each number by 2^k
- Round the result
- Find the integer binary representation

You can either do this directly in signed, or convert as an unsigned number then convert back to signed (invert bits and add 1).

x	int(x.2^6)	binary	int(x.2^6)/2^6	Abs Err
4.3421	278	0100010110	4.343750	0.001650
-6.1150	-391	1001111001	-6.109375	0.005625
0.7721	49	0000110001	0.765625	0.006475
1.9793	127	0001111111	1.984375	0.005075
-2.4400	-156	1101100100	-2.437500	0.002500
0.0391	3	0000000011	0.046875	0.007775

Answer 3

Multiplying two numbers yields a result with a width the sum of the two operands. For fixed point numbers, the number of fractional bits is the sum of the original number of fractional bits.

We can determine the range of fixed point numbers by looking at the number of integer bits.

Op1	Op2	Result	Wordlength	Range
4.6	4.6	8.12	20 bits	0 to 255.999756
3.9	1.11	4.20	24 bits	0 to 15.9999....
4.12	8.8	12.20	32 bits	-2048 to +2047.9999...
1.15	1.15	2.30	32 bits	-2 to +1.9999...

Answer 4

We can look at the weight of each fractional bit until we reach a weight less than 0.0001:

Bit	Weight
2^{-1}	0.5
2^{-2}	0.25
2^{-3}	0.125
2^{-4}	0.0625
2^{-5}	0.03125
2^{-6}	0.015625
2^{-7}	0.0078125
2^{-8}	0.00390625
2^{-9}	0.001953125
2^{-10}	0.0009765625
2^{-11}	0.00048828125
2^{-12}	0.000244140625
2^{-13}	0.0001220703125
2^{-14}	**0.00006103515625**

So we need 14 fractional bits to attain that accuracy.

Alternatively, you can use the approach from Q2.

Which is: $0.0001 \times 2^{??} \geq 1.0$ (Thus need to x by 10,000)

Thus: $2^{??} \geq 10{,}000 \Rightarrow 2^{14} \geq 10{,}000$

So we need 14 fractional bits to attain that accuracy.

Answer 5

(a) +3.965 to -4.381. Firstly, -4.381 requires 4 integer bits, leaving 12 fractional bits. (even though the +ve values only need 3 bits)

(b) +0.99 to -0.99. So need *+0.something* to *-1*, need at least 1 integer bit (will be 0 for +ve numbers and 1 for –ve numbers). Hence 1 integer and 15 fractional bits.

(c) Firstly for the summation: Adding two numbers does not change the bit widths, EXCEPT that an overflow could occur. Hence the size of the result from just the addition should be 5.12.

For the multiplication, we add the individual bit widths. So ***p*** should have a 6.27 format.

Ans = 6 integer and 27 fractional bits (33 bits in total).

If we wanted a result (say ***res***) with just 16 bits, we could throw away 17 fractional bits, leaving 6 integer and 10 fractional bits.

However, looking at the possible range of the numbers for ***p***, we see that they can range from:

(-4.381 + -4.381) * 0.99 = -8.67438

up to

(-4.381 + -4.381) * -0.99 = 8.67438

Hence we only need 5 integer bits. So, a better answer for ***res*** would be 5.11, which could be achieved using:

assign res = p[31:16];